Lab 2: Apache Spark & RDDs – Practical Guide- there was no evalution

Getting Started

Before running these examples, ensure you have your terminal open and have started the Spark Shell.

1. Open Terminal.
2. Type: spark-shell
3. Wait for the scala> prompt.

(Note: In the Spark Shell, a SparkContext is already created for you and stored in the variable sc. You do not need to create it manually.)

Part 1: Creating RDDs (Resilient Distributed Datasets)

There are two main ways to create an RDD: using parallelize on data you already have, or loading an external file.

Example A: Parallelizing an Existing Collection

This converts a standard Scala array into a distributed Spark dataset.

The Code:

Scala
```
// 1. Create a standard Scala array
val data = Array(1, 2, 3, 4, 5)

// 2. Convert it into an RDD using sc.parallelize
val distData = sc.parallelize(data)

// 3. Verify it exists (Optional)
distData.collect()
```

Example B: Referencing an External Dataset

This loads a text file into Spark.

Prerequisite: You need a file to load. Run this command in a **separate** terminal window to create a dummy file: echo "Hello Spark" > simple.txt

The Code:

Scala
```
// Load the text file into an RDD called 'lines'
// Replace "simple.txt" with the actual path if your file is elsewhere
val lines = sc.textFile("simple.txt")
```

```
// View the content of the RDD
lines.collect()
```

(Source: Lab 2 PDF, Page 14)

Part 2: RDD Operations (Transformations & Actions)

Spark operations fall into two categories:

- **Transformations:** Create a new RDD from an existing one (Lazy – nothing happens until you ask for a result).
- **Actions:** Return a value to the driver program (Triggers the computation).

Practical Task: Filter and First

In this example, we will load a file and filter for lines that contain a specific word.

The Code:

Scala
```
// 1. Create an RDD from the README file (assuming you have one, or use simple.txt)
val lines = sc.textFile("simple.txt")

// 2. TRANSFORMATION: Filter lines that contain the word "Spark"
// (Note: This is 'lazy'. No data is processed yet.)
val filteredLines = lines.filter(line => line.contains("Spark"))

// 3. ACTION: Retrieve the first result
// (This triggers the filter operation to actually run.)
filteredLines.first()
```

(Source: Lab 2 PDF, Page 17)

Part 3: Working with Parallelized Collections

We can perform mathematical operations on distributed data, such as summing up numbers using reduce.

The Code:

Scala
```
// 1. Create the dataset
val data = Array(1, 2, 3, 4, 5)
val distData = sc.parallelize(data)

// 2. Use 'reduce' to add up all elements
// (a, b) => a + b means "take two numbers and add them, repeat until done"
val sum = distData.reduce((a, b) => a + b)
```

```
// 3. Print the result
println("Sum: " + sum)
```

(Source: Lab 2 PDF, Page 18)

Part 4: Controlling Partitions

When you create an RDD, Spark divides the data into "partitions". You can manually control how many partitions are created.

The Code:

Scala
```
// Create an RDD with exactly 10 partitions
val data = Array(1, 2, 3, 4, 5)
val partitionedData = sc.parallelize(data, 10)

// Check the number of partitions to verify
partitionedData.getNumPartitions
```

(Source: Lab 2 PDF, Page 19)

Part 5: Standalone Applications (Theory Only)- skip

Important: You do **not** run this inside spark-shell.

The slide on Page 20 explains how to initialize Spark if you were writing a standalone application (like a compiled JAR file) instead of using the interactive shell.

Concept:

When writing a standalone app, you must manually create the SparkConf and SparkContext objects. (In the shell, sc is created automatically, so you skip this).

Reference Code (For documentation only):

Scala
```
import org.apache.spark.SparkContext
import org.apache.spark.SparkConf

// 1. Configure the application
val conf = new SparkConf().setAppName("My App").setMaster("local")

// 2. Create the Context
val sc = new SparkContext(conf)
```

(Source: Lab 2 PDF, Page 20)

```
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/02/04 07:23:16 WARN Utils: Your hostname, Raghads-MacBook-Air.local, resolves to a loopback address: 127.0.0.1; using 17
26/02/04 07:23:16 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.1.1
      /_/

Using Scala version 2.13.17 (OpenJDK 64-Bit Server VM, Java 21.0.10)
Type in expressions to have them evaluated.
Type :help for more information.
26/02/04 07:23:19 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java class
Spark context Web UI available at http://172.20.10.3:4040
Spark context available as 'sc' (master = local[*], app id = local-1770179000215).
Spark session available as 'spark'.

scala> // 1. Create a standard Scala array
     | val data = Array(1, 2, 3, 4, 5)
     |
val data: Array[Int] = Array(1, 2, 3, 4, 5)

scala> // 2. Convert it into an RDD using sc.parallelize
     | val distData = sc.parallelize(data)
[    |
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
val distData: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at <console>:2

scala> // 3. Verify it exists (Optional)
[    | distData.collect()
     |
val res0: Array[Int] = Array(1, 2, 3, 4, 5)

scala> // Load the text file into an RDD called 'lines'
     | // Replace "simple.txt" with the actual path if your file is elsewhere
     | val lines = sc.textFile("simple.txt")
     |
[    | // View the content of the RDD
     | lines.collect()
     |
val lines: org.apache.spark.rdd.RDD[String] = simple.txt MapPartitionsRDD[2] at textFile at <console>:3
val res1: Array[String] = Array(Hello Spark)

scala> Scala
     | // 1. Create an RDD from the README file (assuming you have one, or use simple.txt)
     | val lines = sc.textFile("simple.txt")
[    |
     | // 2. TRANSFORMATION: Filter lines that contain the word "Spark"
     | // (Note: This is 'lazy'. No data is processed yet.)
     | val filteredLines = lines.filter(line => line.contains("Spark"))
     |
     | // 3. ACTION: Retrieve the first result
     | // (This triggers the filter operation to actually run.)
     | filteredLines.first()
     |
     |
       ^
       error: not found: value Scala
```

```
scala> // 1. Create an RDD from the README file (assuming you have one, or use simple.txt)
     | val lines = sc.textFile("simple.txt")
     |
     | // 2. TRANSFORMATION: Filter lines that contain the word "Spark"
     | // (Note: This is 'lazy'. No data is processed yet.)
     | val filteredLines = lines.filter(line => line.contains("Spark"))
     |
     | // 3. ACTION: Retrieve the first result
     | // (This triggers the filter operation to actually run.)
     | filteredLines.first()
     |
     |
val lines: org.apache.spark.rdd.RDD[String] = simple.txt MapPartitionsRDD[4] at textFile at <console>:2
val filteredLines: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[5] at filter at <console>:6
val res3: String = Hello Spark

scala> // 1. Create the dataset
     | val data = Array(1, 2, 3, 4, 5)
     | val distData = sc.parallelize(data)
     |
     | // 2. Use 'reduce' to add up all elements
     | // (a, b) => a + b means "take two numbers and add them, repeat until done"
     | val sum = distData.reduce((a, b) => a + b)
     |
     | // 3. Print the result
     | println("Sum: " + sum)
     |
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
Sum: 15
val data: Array[Int] = Array(1, 2, 3, 4, 5)
val distData: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[6] at parallelize at <console>:3
val sum: Int = 15

scala> // Create an RDD with exactly 10 partitions
     | val data = Array(1, 2, 3, 4, 5)
     | val partitionedData = sc.parallelize(data, 10)
     |
     | // Check the number of partitions to verify
     | partitionedData.getNumPartitions
     |
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
val data: Array[Int] = Array(1, 2, 3, 4, 5)
val partitionedData: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[7] at parallelize at <console>:3
val res5: Int = 10

scala> 26/02/04 08:49:15 WARN HeartbeatReceiver: Removing executor driver with no recent heartbeats: 393191 ms exceeds timeout 120000 ms
26/02/04 08:49:15 WARN SparkContext: Killing executors is not supported by current scheduler.
26/02/04 08:49:22 ERROR Inbox: Ignoring error
org.apache.spark.SparkException: Exception thrown in awaitResult:
        at org.apache.spark.util.SparkThreadUtils$.awaitResult(SparkThreadUtils.scala:53)
        at org.apache.spark.util.ThreadUtils$.awaitResult(ThreadUtils.scala:359)
```